Listing 8.4 `document_visualizing_embeddings.py` (relevant sections)

```
load_dotenv()      #1
api_key = os.getenv('OPENAI_API_KEY')
if not api_key:
    raise ValueError("No API key found. Please check your .env file.")
client = OpenAI(api_key=api_key)     #1

def get_embedding(text, model="text-embedding-ada-002"):     #2
    text = text.replace("\n", " ")
    return client.embeddings.create(input=[text],
              model=model).data[0].embedding               #2

# Sample documents (omitted)

embeddings = [get_embedding(doc) for doc in documents]    #3
print(embeddings_array.shape)

embeddings_array = np.array(embeddings)    #4

pca = PCA(n_components=3)   #5
reduced_embeddings = pca.fit_transform(embeddings_array)
```

#1 Join all the items on the string ', '.
#2 Uses the OpenAI client to create the embedding
#3 Generates embeddings for each document of size 1536 dimensions
#4 Converts embeddings to a NumPy array for PCA
#5 Applies PCA to reduce dimensions to 3 for plotting

When a document is embedded using an OpenAI model, it transforms the text into a vector with dimensions of 1536. We can't visualize this number of dimensions, so we use a dimensionality reduction technique via principal component analysis (PCA) to convert the vector of size 1536 to 3 dimensions.

Figure 8.6 shows the output generated from running the file in VS Code. By reducing the embeddings to 3D, we can plot the output to show how semantically similar documents are now grouped.